

Strathmore University

@iLabAfrica Centre

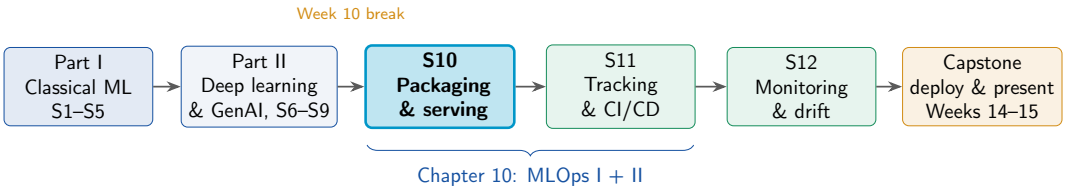
Session 10: MLOps I
Model Packaging & Serving

Javier Serrano

Applied Machine Learning

Master in Data Science & Analytics

Where this session sits in the course

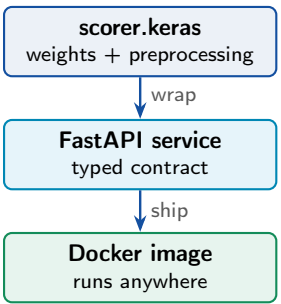


Takeaway: Today we stop asking *"is the model good?"* and start asking *"can anyone call it, safely, at 2 a.m. on salary day?"*

Learning outcomes

By the end of this session you will be able to:

1. **Define** a model artefact that carries its own preprocessing, threshold and metadata.
2. **Choose** a serving pattern from latency, freshness and cost constraints.
3. **Build** a typed, tested FastAPI scoring service.
4. **Containerise** it with a small, secure, reproducible Docker image.
5. **Expose** the service safely to LLM agents as a tool.

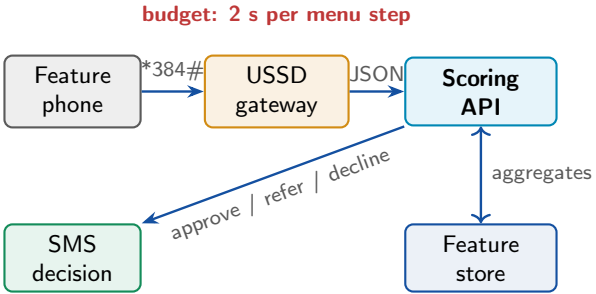


The running case: thin-file scoring over USSD

The same Nairobi credit scorer we have built since Session 1

Case study: Nairobi thin-file credit scorer

- Borrowers with no bureau history; mobile-money behaviour only.
- Default rate $\pi \approx 4\%$.
- Costs: KES 10,000 per missed default, KES 800 per wrongly declined good borrower.
- Channel: a USSD menu on a feature phone.



Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

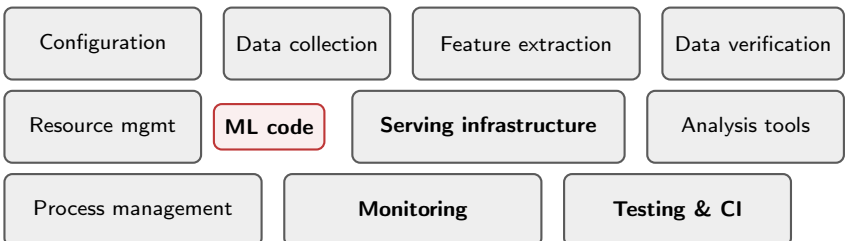
Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

The model is the smallest box

Hidden technical debt in ML systems [Sculley et al., 2015]



- The training code you wrote in Sessions 1–9 is the red box.
- Sessions 10–12 build the three bold boxes around it.

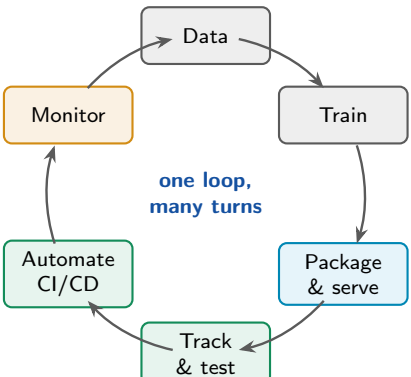
What changes when a model leaves the notebook

Same weights, a completely different environment

Notebook		Production service
Clean `DataFrame`	⟶	Untrusted JSON from strangers
Runs once, when you press run	⟶	Runs 24/7, thousands of callers
Latency irrelevant	⟶	p_{99} latency is a contract
`model_final_v3.keras`	⟶	Immutable, traceable version
Failure: re-run the cell	⟶	Failure: a borrower is harmed

The MLOps lifecycle

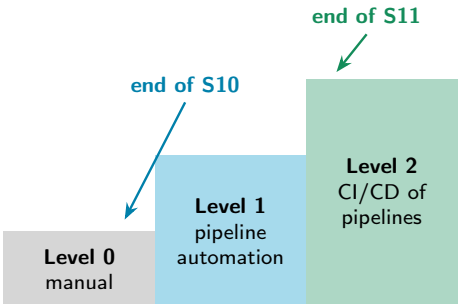
Where Sessions 10, 11 and 12 sit in the loop



- **Cyan**: Session 10 (today).
- **Green**: Session 11.
- **Amber**: Session 12.
- The loop never ends: monitoring triggers retraining, retraining triggers a new package.

MLOps maturity levels

After Google Cloud's continuous-delivery framing [Google Cloud, 2020]



- **Level 0**: a person trains, exports and deploys by hand.
- **Level 1**: retraining runs as a pipeline (continuous training).
- **Level 2**: the pipeline itself is tested and deployed by CI/CD.

Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

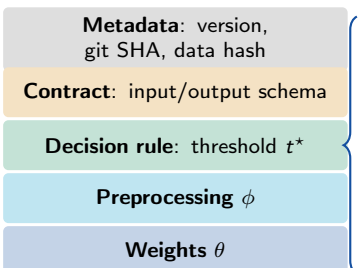
Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

What exactly is "the model"?

Weights alone cannot make a decision



The deployable unit is the whole decision function

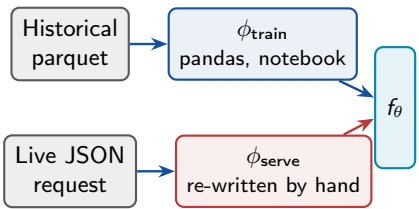
$$\delta(x) \;=\; \mathbb{1}\big[\, f_\theta\big(\phi(x)\big) \geq t^\star \big]\,,$$

not the weights θ.

- Change any layer and you have a *new model version*.
- All five layers must travel together, or none should.

Training–serving skew

The most common silent failure of deployed models



$\phi_{\text{serve}} \neq \phi_{\text{train}}$

$\Rightarrow f_\theta$ sees inputs from a distribution it never saw

Pitfall: skew passes every offline test

Validation used ϕ_{train}, so AUC looks fine. Typical causes: a different log base, a missing clip, a new mobile-money region code, a mean imputed with production data.

Cure: put ϕ inside the graph

Keras preprocessing layers freeze training statistics into the artefact

```
import keras
from keras import layers

NUM = ["tenure_months", "avg_monthly_inflow_kes", "txn_count_90d", "ontime_repayments_12m"]
inputs = {c: keras.Input(shape=(1,), name=c) for c in NUM}
inputs["region"] = keras.Input(shape=(1,), dtype="string", name="region")

norm = {c: layers.Normalization(name=f"norm_{c}") for c in NUM}
for c in NUM:
    norm[c].adapt(train_df[[c]].to_numpy())          # mean/var become weights
region = layers.StringLookup(output_mode="one_hot", num_oov_indices=1)
region.adapt(train_df["region"].to_numpy())          # unseen codes -> OOV bucket

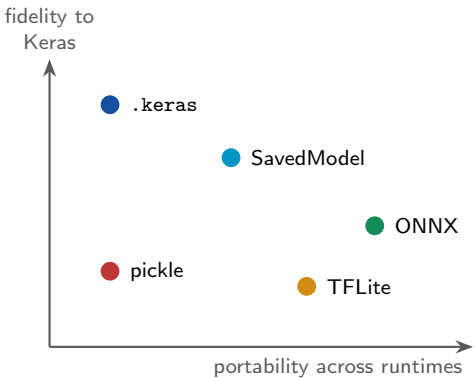
x = layers.Concatenate()([norm[c](inputs[c]) for c in NUM] + [region(inputs["region"])])
x = layers.Dense(32, activation="relu")(x)
pd_out = layers.Dense(1, activation="sigmoid", name="pd")(x)
model = keras.Model(inputs, pd_out, name="thinfile_scorer")
```

Key idea

The serving code now sends *raw* fields. There is no second implementation of ϕ to drift.

Serialisation formats

Trade fidelity to the framework against portability across runtimes



- `.keras`: Python + Keras serving (our FastAPI).
- **SavedModel**: TF Serving, TFLite conversion.
- **ONNX**: runtime-neutral; check op coverage.
- **TFLite**: phones and agent-banking tablets.
- **pickle**: executes code on load. Never accept one from outside.

Save, reload, and prove the round trip

A serialisation you have not reloaded is a serialisation you have not tested

```python
import numpy as np, keras

model.save("artefacts/scorer.keras")          # Keras-native, used by our API
model.export("artefacts/scorer_savedmodel")   # SavedModel, for TF Serving / TFLite

reloaded = keras.saving.load_model("artefacts/scorer.keras")   # safe_mode=True by default

probe = {k: v[:256] for k, v in X_val.items()}              # real validation rows
np.testing.assert_allclose(model(probe), reloaded(probe), atol=1e-6)
```

Key idea

The round-trip assertion becomes a unit test in Session 11. It runs on every commit.

Pitfall

`safe_mode=True` refuses to deserialise arbitrary `Lambda` code. Prefer built-in layers over lambdas.

The threshold belongs in the package

Asymmetric costs move the decision boundary far from 0.5

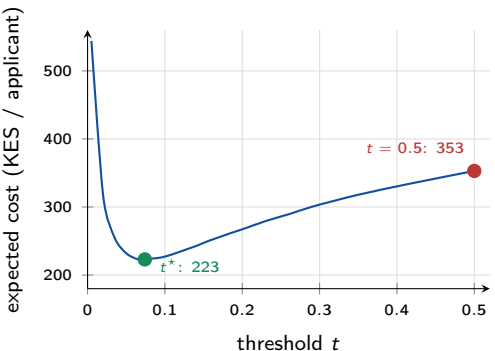
Decline when the expected cost of approving exceeds that of declining:

$$p\,C_{FN} > (1-p)\,C_{FP}$$

$$\iff \quad p > t^\star = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

$$t^\star = \frac{800}{10\,800} \approx 0.074$$

- Valid only if p is **calibrated** (Session 3).
- Ship $t^\star$ and the cost matrix inside the artefact.



A metadata sidecar travels with the weights

The seed of the lineage record that MLflow will formalise in Session 11

```python
import json, subprocess, hashlib

def sha256(path):
    return hashlib.sha256(open(path, "rb").read()).hexdigest()

meta = {
  "model_name": "thinfile-scorer",
  "version": "1.3.0",
  "git_sha": subprocess.check_output(
      ["git", "rev-parse", "--short", "HEAD"]).decode().strip(),
  "data_sha256": sha256("data/snapshots/2026-09-30.parquet"),
  "features": NUM + ["region"],
  "threshold": 0.074,
  "cost_matrix_kes": {"fn": 10000, "fp": 800},
  "val_metrics": {"auc": 0.81, "cost_kes": 223},  # [PLACEHOLDER]
  "tf_version": keras.version(),
}
json.dump(meta, open("artefacts/metadata.json", "w"), indent=2)
```

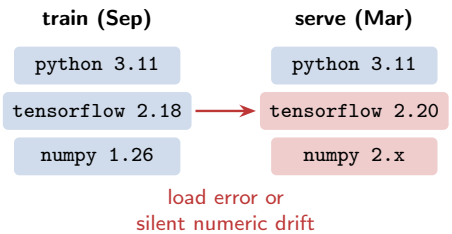
```
artefacts/
├── scorer.keras
├── scorer_savedmodel/
└── metadata.json
```

Key idea

Anyone holding the folder can answer: *which code, which data, which threshold?*

Pin the environment, not just the model

"It worked on my laptop" is a dependency-drift bug



- Declare direct dependencies in `requirements.in`.
- Compile an **exact, hashed** lock file.
- Install only from the lock, everywhere.

```
uv pip compile requirements.in \
    --generate-hashes \
    -o requirements.lock
pip install --require-hashes \
    -r requirements.lock
```

Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

Four ways to serve predictions

Chosen by latency, feature freshness and where the compute lives

Batch

Score everyone on a schedule.

Nightly credit-limit refresh.

Latency: hours.

Online

Score one request, now.

USSD loan application.

Latency: ms.

Streaming

Score events as they flow.

Fraud on mobile-money transfers.

Latency: ms, continuous.

Edge

Score on the device.

Agent tablet with no signal.

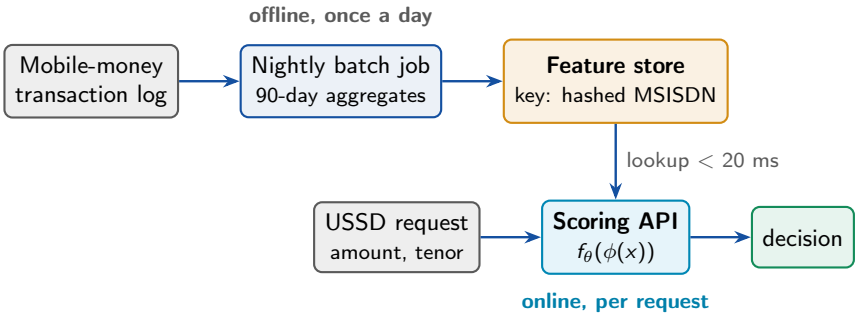
Latency: ms, offline.



stricter latency, fresher features

Our case is hybrid

Heavy aggregates in batch, the decision online



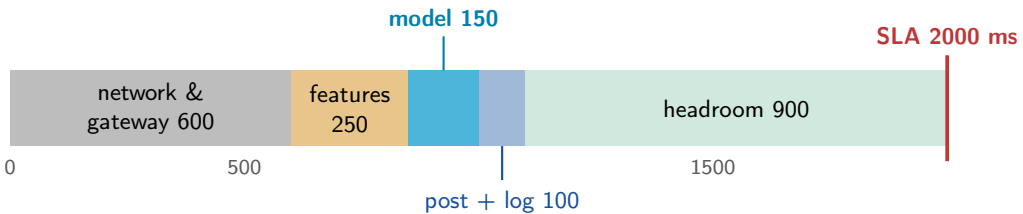
Key idea

Move every computation that does not depend on the request out of the request path.

A latency budget

Decompose the SLA before writing any code

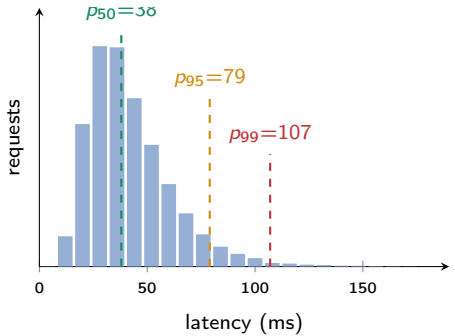
$$T_{\text{e2e}} = T_{\text{net}} + T_{\text{feat}} + T_{\text{inf}} + T_{\text{post}} \leq T_{\text{SLA}}$$



- The model gets **150 ms**: this rules out a 7B-parameter LLM in the request path.
- Headroom absorbs retries and the long tail (next slide).

Tail latency is what users feel

Report percentiles, never the mean



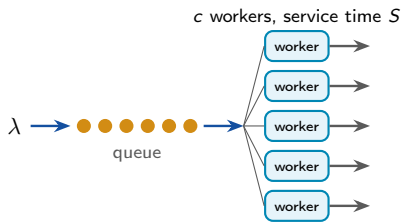
Fan-out amplifies the tail. If a request touches n services, each fast with probability q:

$$\Pr(\text{no slow call}) = q^n$$

With $q = 0.99$ and $n = 5$: $0.99^5 \approx 0.95$, so **1 in 20** requests hits someone's p_{99} [Dean & Barroso, 2013].

How many workers? Little's law

Capacity planning for the month-end salary spike



Little's law: $L = \lambda W$ (in-flight = rate $\times$ time).

Offered load and worker count:

$$a = \lambda \bar{S}, \qquad c \geq \left\lceil \frac{a}{\rho_{\max}} \right\rceil$$

Example (*placeholder*): $\lambda = 120$ req/s, $\bar{S} = 25$ ms, $\rho_{\max} = 0.6$:

$$a = 3.0 \;\Rightarrow\; c = 5 \text{ workers.}$$

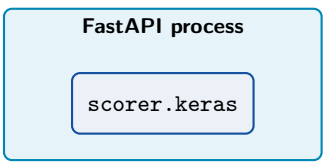
Key idea

Keep utilisation $\rho = a/c$ well below 1: queueing delay explodes as $\rho \to 1$.

Model-in-service or model-as-a-service?

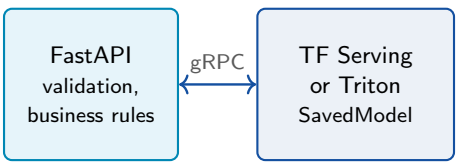
Embed the model in the API, or call a dedicated model server

Model-in-service (today)



simple, one image, one deploy
model and API scale together

Model-as-a-service



independent scaling, GPU batching,
multi-model; one more network hop

Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

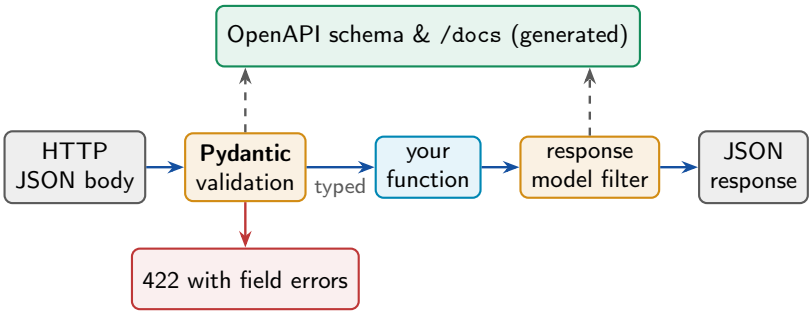
Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

Why FastAPI

Type hints become validation, serialisation and documentation



- ASGI server (`uvicorn`); sync and async endpoints side by side.
- The schema you write once is the contract for callers, tests and agents.

Contract first: the request schema

Reject impossible inputs before they reach the model

```python
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

Region = Literal["nairobi", "central", "coast", "eastern",
                 "north_eastern", "nyanza", "rift_valley", "western"]

class Applicant(BaseModel):
    model_config = ConfigDict(extra="forbid")          # unknown fields -> 422

    msisdn_hash: str = Field(pattern=r"^[0-9a-f]{64}$") # salted SHA-256
    tenure_months: int = Field(ge=0, le=480)
    avg_monthly_inflow_kes: float = Field(ge=0, le=5_000_000)
    txn_count_90d: int = Field(ge=0)
    ontime_repayments_12m: int = Field(ge=0, le=365)
    region: Region
```

Key idea

Bounds encode domain knowledge: the first defence against garbage in.

Kenyan context

DPA 2019 minimisation: the API never sees the raw phone number.

The response is an audit record

Every field answers a question a regulator or customer may ask

```python
class ScoreResponse(BaseModel):
    request_id: str                    # joins logs
    pd: float | None = Field(ge=0.0, le=1.0)
    decision: Literal["approve", "refer", "decline"]
    threshold: float                   # t* used here
    reason_codes: list[str]            # top drivers
    model_version: str                 # "1.3.0+a1b2c3d"
```

Customer: why was I declined?

Risk: which threshold?

Engineer: which model?

Auditor: show me this call.

- pd is `None` when the case is referred to a human loan officer (e.g. no transaction history).

Load the model once: the lifespan handler

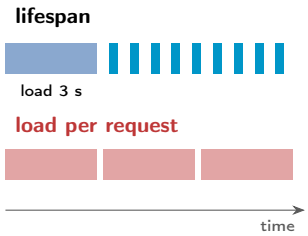
Loading per request turns 20 ms into 3 s

```python
import json
from contextlib import asynccontextmanager
import keras
from fastapi import FastAPI

STATE: dict = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    STATE["model"] = keras.saving.load_model(
        "artefacts/scorer.keras")
    STATE["meta"] = json.load(
        open("artefacts/metadata.json"))
    yield                       # serve requests here
    STATE.clear()               # graceful shutdown

app = FastAPI(title="Thin-file scorer",
              version="1.3.0", lifespan=lifespan)
```



The scoring endpoint

Thin: validate, predict, apply the rule, explain

```python
import numpy as np
from uuid import uuid4

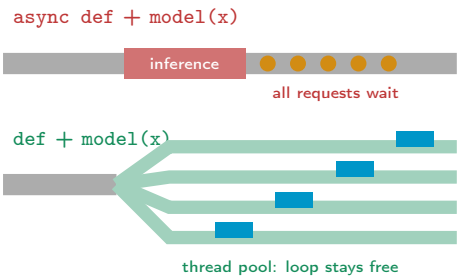
@app.post("/v1/score", response_model=ScoreResponse)
def score(a: Applicant) -> ScoreResponse:              # def, not async def (next slide)
    meta, rid = STATE["meta"], str(uuid4())
    feats = a.model_dump(exclude={"msisdn_hash"})
    base = dict(request_id=rid, threshold=meta["threshold"], model_version=meta["version"])

    if feats["txn_count_90d"] == 0:                    # business rule before the model
        return ScoreResponse(**base, pd=None, decision="refer", reason_codes=["NO_HISTORY"])

    x = {k: np.array([[v]]) for k, v in feats.items()}
    pd_ = float(STATE["model"](x, training=False)[0, 0])   # direct call: no predict()
     overhead
    decision = "decline" if pd_ >= meta["threshold"] else "approve"
    return ScoreResponse(**base, pd=pd_, decision=decision,
                         reason_codes=top_reasons(x))  # attribution helper, app/explain.py
```

def or async def?

CPU-bound inference must never block the event loop



- `async def`: runs *on* the event loop. Use for I/O: databases, HTTP, LLM streams.
- `def`: FastAPI runs it in a thread pool. Use for CPU work like TF inference.
- Mixing them wrongly gives great single-user tests and terrible load tests.

Liveness, readiness and metadata

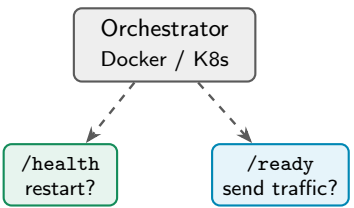
Let the orchestrator ask the right question

```python
from fastapi import Response

@app.get("/health")                    # liveness
def health():
    return {"status": "ok"}

@app.get("/ready")                     # readiness
def ready(response: Response):
    if "model" not in STATE:
        response.status_code = 503
        return {"ready": False}
    return {"ready": True}

@app.get("/metadata")                  # provenance
def metadata():
    return STATE["meta"]
```



- Alive $\neq$ ready: a container loading a model is alive but not ready.

Errors are part of the contract

Status codes tell the caller what to do next

200	**422**	**503**	**500**
scored	invalid input	not ready	our bug
show decision	*caller fixes request*	*retry with back-off*	*page on-call*

- Never return a stack trace to the caller; log it with the `request_id`.
- For USSD, a timeout must fail *closed*: "we will SMS you", never an automatic approval.

Batch endpoints amortise fixed cost

Why `/v1/score/batch` exists for the nightly job

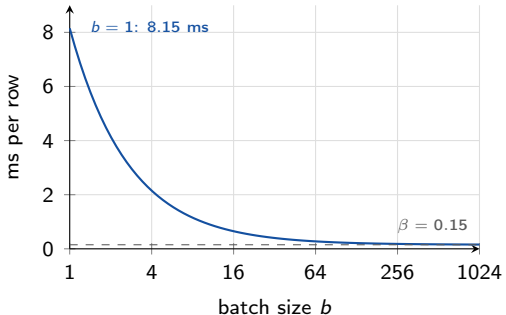
Model call time for a batch of b rows:

$$T(b) = \alpha + \beta\, b$$

Per-row cost:

$$\frac{T(b)}{b} = \frac{\alpha}{b} + \beta \xrightarrow{b \to \infty} \beta$$

- α: dispatch, graph and HTTP overhead.
- Online path: $b = 1$. Batch path: b in the hundreds.



Test the API like a caller would

`TestClient` runs the lifespan, the validation and the endpoint

```python
from fastapi.testclient import TestClient
from app.main import app

GOOD = {"msisdn_hash": "a" * 64, "tenure_months": 26, "avg_monthly_inflow_kes": 18_500.0,
        "txn_count_90d": 143, "ontime_repayments_12m": 9, "region": "nairobi"}

def test_score_contract():
    with TestClient(app) as client:                    # triggers lifespan: model loads
        r = client.post("/v1/score", json=GOOD)
        assert r.status_code == 200
        body = r.json()
        assert 0.0 <= body["pd"] <= 1.0
        assert body["decision"] in {"approve", "refer", "decline"}

def test_rejects_negative_tenure():
    with TestClient(app) as client:
        assert client.post("/v1/score", json={**GOOD, "tenure_months": -3}).status_code == 422
```

Key idea

These tests become the merge gate of the CI pipeline in Session 11.

Running it: processes and threads

Avoid oversubscribing the CPU

4 workers × 1 thread **4 workers × 4 threads**



4 CPU cores in each case

workers × threads ≤ cores

```
# 4 cores: 4 workers x 1 thread
export OMP_NUM_THREADS=1
export TF_NUM_INTRAOP_THREADS=1
uvicorn app.main:app --host 0.0.0.0 \
        --port 8000 --workers 4
```

- Each worker loads its own copy of the model: memory ≈ workers × model size.

Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

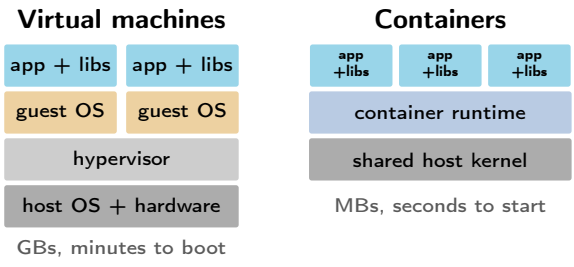
Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

The environment is part of the model

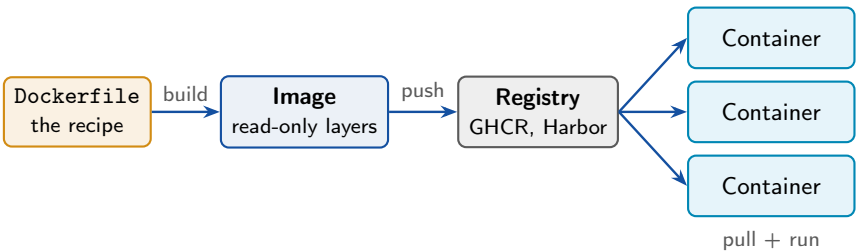
Containers package code, dependencies and runtime together



- Same image on a laptop, a CI runner and a Nairobi data centre.
- Isolation without a guest OS per service.
- The image digest is an **immutable** version of the whole stack.

Image, container, registry

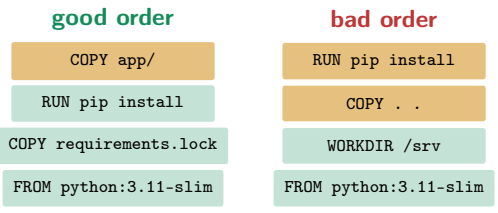
Build once, run many times, anywhere



- **Image**: a template. **Container**: a running instance with its own writable layer.
- Containers are cattle, not pets: replace them, never patch them in place.

Layers and the build cache

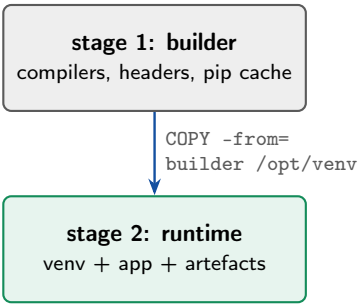
Order instructions from least to most frequently changed



- Each instruction creates a layer; a change invalidates it *and every layer above*.
- Copy the lock file and install first; copy code last.
- Bad order: a one-line edit re-downloads TensorFlow.
- A `.dockerignore` keeps `data/`, `.git` and `.env` out of the build context.

Multi-stage builds

Build tools stay in stage 1; only the runtime ships



only stage 2 becomes the image

```dockerfile
# ---------- stage 1: builder ----------
FROM python:3.11-slim AS builder
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.lock .
RUN pip install --no-cache-dir \
      --require-hashes -r requirements.lock
```

- The builder can be as heavy as it needs to be; it is thrown away.
- A virtual environment is one directory: easy to copy across stages.

Stage 2: the runtime image

Non-root, pinned, self-describing, with a healthcheck

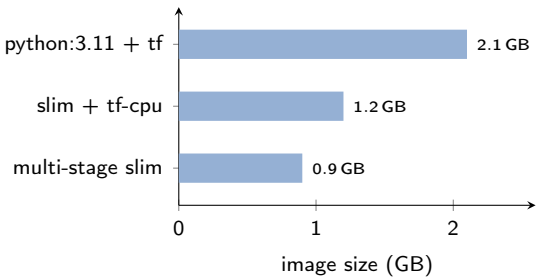
```
# ---------- stage 2: runtime ----------
FROM python:3.11-slim
RUN useradd --create-home --uid 10001 app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH" PYTHONUNBUFFERED=1 \
    OMP_NUM_THREADS=1 TF_NUM_INTRAOP_THREADS=1
WORKDIR /srv
COPY app/ app/
COPY artefacts/ artefacts/
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD python -c "import urllib.request as u; u.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

Key idea

The model is baked into the image: the image digest *is* the release. Session 11 discusses when to fetch it from a registry instead.

Image size matters on real networks

Download time $t = S/B$ on a constrained link



At $B = 10$ Mbit/s:

$$t_{2.1\,\mathrm{GB}} = \frac{2.1 \times 8 \times 10^3}{10} \approx 28 \text{ min}$$

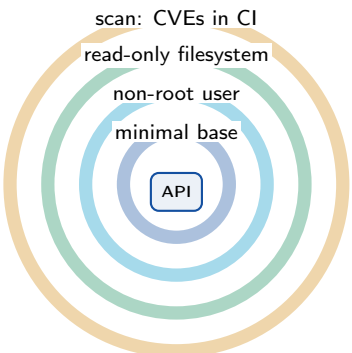
$$t_{0.9\,\mathrm{GB}} \approx 12 \text{ min}$$

Kenyan context

Edge sites and branch servers often pull over metered or shared links. Small images, a local registry mirror and stable base layers cut deploy time and cost.

Hardening the container

Defence in depth for a service that makes credit decisions



- **Minimal base**: `-slim` or distroless; fewer packages, fewer CVEs.
- **Non-root**: `USER app`; a compromise does not own the host.
- **No secrets in images**: layers are forever. Inject at run time.
- **Scan** every build (e.g. Trivy) and fail on critical CVEs.
- **Pin by digest**: `python:3.11-slim@sha256:...`

Build, tag, run

A tag should tell you exactly what is inside

`ghcr.io/ilabafrica` `/thinfile-scorer` `:1.3.0` `-a1b2c3d`

registry / org repository model version git SHA

```
IMG=ghcr.io/ilabafrica/thinfile-scorer:1.3.0-$(git rev-parse --short HEAD)
docker build -t "$IMG" .
docker run -d --name scorer -p 8000:8000 --read-only --tmpfs /tmp \
       --cpus 2 --memory 2g -e LOG_LEVEL=info "$IMG"
curl -s localhost:8000/ready && curl -s -X POST localhost:8000/v1/score \
       -H 'Content-Type: application/json' -d @tests/fixtures/applicant.json
docker push "$IMG"
```

Pitfall: `:latest`

`latest` is a moving pointer, not a version. Never deploy it to production.

Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

What changes when the model is an LLM

Same packaging discipline, different physics

	Tabular scorer (today)	LLM / RAG service (Session 9)
Latency	10–50 ms	1–20 s
Hardware	CPU, 100 MB	GPU, 10s of GB
Output	one JSON object	token stream (SSE)
Unit cost	negligible	per token
Determinism	exact	sampled, prompt-sensitive

- Keep LLMs **out of the 2-second decision path**; use them where latency is tolerated: officer explanations, collections messaging, document triage.

Streaming responses for LLM endpoints

Here `async def` is the right choice: the work is I/O

```python
from fastapi.responses import StreamingResponse

class ExplainRequest(BaseModel):
    request_id: str                          # the scoring call being explained
    language: Literal["en", "sw"] = "en"     # English or Kiswahili for the officer

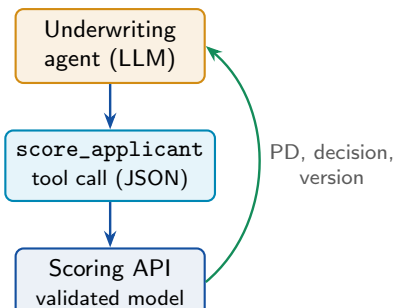
@app.post("/v1/explain")
async def explain(req: ExplainRequest):
    record = await decision_log.get(req.request_id)          # reason codes, never raw PII
    async def events():
        async for chunk in llm.stream(build_prompt(record, req.language)):  # any async LLM SDK
            yield f"data: {chunk}\n\n"                        # Server-Sent Events framing
    return StreamingResponse(events(), media_type="text/event-stream")
```

Key idea

The LLM verbalises reason codes the scorer already produced. It explains the decision; it does not make it.

Your scorer as an agent tool

The Pydantic contract doubles as the tool schema




```
score_tool = {
  "name": "score_applicant",
  "description": ("Probability of default for a "
     "thin-file applicant. Returns decision, "
     "reason codes and model version."),
  "input_schema": Applicant.model_json_schema(),
}
```

- One schema: API, tests and agent never disagree.
- The agent orchestrates; the audited model decides.

Roadmap

From notebook to service

Packaging the model artefact

Serving patterns and latency

Building the API with FastAPI

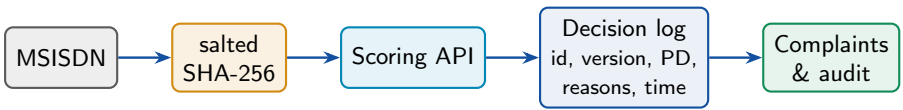
Containerising with Docker

Serving LLM and agent workloads

Governance and wrap-up

Regulation shapes the serving design

Kenya DPA 2019 and the CBK Digital Credit Providers Regulations 2022



Kenyan context

- **Minimisation** (DPA s.25): the service receives a pseudonymous key, not a phone number.
- **Automated decisions** (DPA s.35): the `refer` path and a human review route are design features, not afterthoughts.
- **Complaints** (CBK DCP 2022): every decision is reconstructable from its `request_id`.

Pitfalls to avoid

Each of these has taken down a real deployment

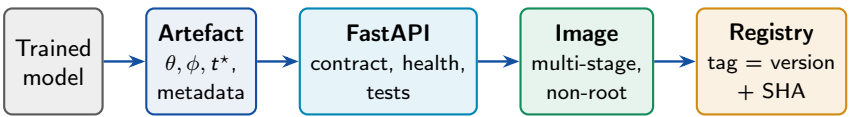
Five production pitfalls

1. Re-implementing preprocessing in the API (skew).
2. Loading the model inside the request handler.
3. CPU-bound inference in an `async def` endpoint.
4. Shipping `:latest`, secrets or `data/` in the image.
5. Failing *open*: a timeout that auto-approves.



Summary

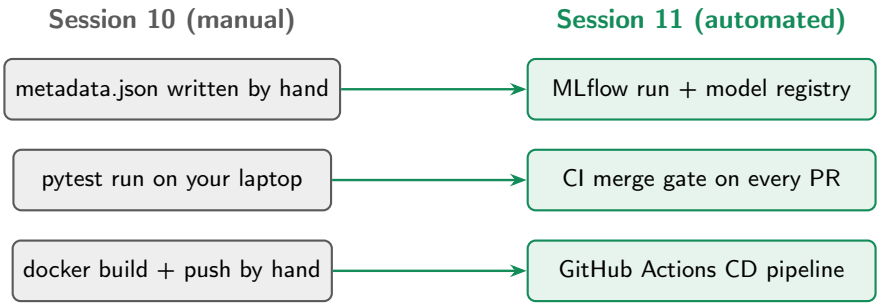
Session in one picture: From a notebook model to a shippable, auditable service



- Package the *decision function*, not the weights.
- Budget latency first; percentiles, not means.
- The schema is the contract for callers, tests and agents.
- The image digest is the release.

Next session: from manual to automatic

Every manual step today becomes a pipeline step in Session 11



References

Chapter 10 of the course book develops every slide in prose

- **Huyen** (2022), *Designing Machine Learning Systems*, Ch. 7: deployment, batch vs online prediction, model compression.
- **Géron** (2019), *Hands-On ML*, 2nd ed., Ch. 19 (3rd ed., Ch. 19): saving models, TF Serving, deployment to mobile and the cloud.
- **Sculley et al.** (2015), "Hidden technical debt in machine learning systems", NeurIPS.
- **Dean & Barroso** (2013), "The tail at scale", *Communications of the ACM*.
- **Documentation**: FastAPI (lifespan, testing, concurrency); Docker (multi-stage builds, build cache); Keras (preprocessing layers, serialisation).